SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME:Database Management Systems

COURSE CODE:CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS Total Marks: 50

Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	**5**
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	**5**
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	**5**
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	**5**
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	**5**
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	**5**
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	**5**
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	**5**
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	**5**
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	**5**

Code of Conduct:

I __M Dhruthika(192572020)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1:Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.

Relational Algebra Operations on Employee and Department

Assume:**Employee(Emp_ID, Emp_Name, Salary, Dept_ID)Department(Dept_ID, Dept_Name)**

1. **Selection (σ)**
    1. Used to select specific **rows** from a relation.
    2. Example: σ Salary > 50000 (Employee)
    3. Retrieves employees whose salary is greater than 50,000.
2. **Projection (π)**
    1. Used to select specific **columns** from a relation.
    2. Example: π Emp_Name, Salary (Employee)
    3. Displays only employee names and salaries.
3. **Union (∪)**
    1. Combines the records of two compatible relations.
    2. Example: Employee1 ∪ Employee2
    3. Returns all records from both relations without duplicates.
4. **Difference (−)**
    1. Returns records that are present in the first relation but not in the second.
    2. Example: Employee1 − Employee2
5. **Cartesian Product (×)**
    1. Combines every record of Employee with every record of Department.
    2. Example: Employee × Department
    3. Produces all possible combinations of records.
6. **Join (⋈)**
    1. Combines related records from Employee and Department using a common attribute.
    2. Example: Employee ⋈ Employee.Dept_ID = Department.Dept_ID Department
    3. Retrieves employee details along with their department details.

Q2:Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

SQL DDL Statements to Create STUDENT and DEPARTMENT Tables

1. **Create DEPARTMENT table**

```
CREATE TABLE DEPARTMENT (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50) NOT NULL UNIQUE
);
```

1. **Create STUDENT table**

```
CREATE TABLE STUDENT (
    SID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    DOB DATE NOT NULL,
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);
```

Constraints Used

- **PRIMARY KEY** → Uniquely identifies each student and department.
- **NOT NULL** → Ensures Name, DOB, and DeptName cannot be empty.
- **UNIQUE** → Ensures department names are not duplicated.
- **FOREIGN KEY** → Connects STUDENT.DeptID with DEPARTMENT.DeptID.
- **DATE** → Stores the student's date of birth.

Q3: Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

SQL Queries Using GROUP BY, HAVING and Aggregate Functions

Assume the **Sales** table contains:

Sales(SaleID, Product, Category, Quantity, Amount)

1. **COUNT() – Count the number of sales**

```
SELECT Category, COUNT(*) AS Total_Sales
FROM Sales
GROUP BY Category;
```

1. **SUM() – Find total sales amount**

```
SELECT Category, SUM(Amount) AS Total_Amount
FROM Sales
```

GROUP BY Category;

1. **AVG() – Find average sales amount**

SELECT Category, AVG(Amount) AS Average_Amount

FROM Sales

GROUP BY Category;

1. **MAX() – Find maximum sale amount**

SELECT Category, MAX(Amount) AS Maximum_Amount

FROM Sales

GROUP BY Category;

1. **MIN() – Find minimum sale amount**

SELECT Category, MIN(Amount) AS Minimum_Amount

FROM Sales

GROUP BY Category;

1. **HAVING – Display categories with total sales above 50,000**

SELECT Category, SUM(Amount) AS Total_Amount

FROM Sales

GROUP BY Category

HAVING SUM(Amount) > 50000;

Q4:Write a correlated sub-query to find employees earning more than the average salary of their own department.

Correlated Sub-query

Assume the table is:

EMPLOYEE(EmpID, EmpName, DeptID, Salary)

```
SELECT E.EmpID, E.EmpName, E.DeptID, E.Salary
FROM EMPLOYEE E
WHERE E.Salary > (
   SELECT AVG(E2.Salary)
   FROM EMPLOYEE E2
   WHERE E2.DeptID = E.DeptID
);
```

Q5:Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

JOIN Operations on Employee and Project Tables

Assume:

- **EMPLOYEE(EmpID, EmpName, DeptID)**
- **PROJECT(ProjectID, ProjectName, EmpID)**

1. **INNER JOIN**

Returns employees who are assigned to a project.

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM Employee E
INNER JOIN Project P
ON E.EmpID = P.EmpID;
```

2.LEFT OUTER JOIN

Returns all employees, including those without a project.

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM Employee E
LEFT JOIN Project P
ON E.EmpID = P.EmpID;
```

3.RIGHT OUTER JOIN

Returns **all projects**, including projects with no assigned employee.

```
SELECT E.EmpID, E.EmpName, P.ProjectName

FROM Employee E

RIGHT JOIN Project P

ON E.EmpID = P.EmpID;
```

4.FULL OUTER JOIN

1. Returns **all employees and all projects**.
2. Unmatched values are shown as `NULL`.

```
SELECT E.EmpID, E.EmpName, P.ProjectName

FROM Employee E

FULL OUTER JOIN Project P

ON E.EmpID = P.EmpID;
```

5.CROSS JOIN

1. Combines every employee with every project.
2. Produces all possible combinations.

```
SELECT E.EmpName, P.ProjectName

FROM Employee E

CROSS JOIN Project P;
```

Q6:Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

Assume the tables are:

- **DEPARTMENT(DeptID, DeptName)**
- **EMPLOYEE(EmpID, EmpName, DeptID, Salary)**

1. **Create the view**

```
CREATE VIEW DeptSalaryView AS

SELECT D.DeptName,

    COUNT(E.EmpID) AS Total_Employees,

    AVG(E.Salary) AS Average_Salary
```

```
FROM DEPARTMENT D

LEFT JOIN EMPLOYEE E

ON D.DeptID = E.DeptID

GROUP BY D.DeptName;
```

1. **Query the view**

```
SELECT *

FROM DeptSalaryView;
```

1. **Query departments with average salary above 50,000**

```
SELECT *

FROM DeptSalaryView

WHERE Average_Salary > 50000;
```

Q7:Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

Relational Algebra Expression

Assume the relations are:

- **STUDENT(SID, Name)**
- **COURSE(CID, CourseName)**
- **ENROLL(SID, CID)**

To find students enrolled in all courses offered, use the division (÷) operator:

$\pi_{SID,CID}(ENROLL) \div \pi_{CID}(COURSE)$

Then get the student names:

$\pi_{Name}(STUDENT \bowtie (\pi_{SID,CID}(ENROLL) \div \pi_{CID}(COURSE)))$

Q8:Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

SQL DML Statements with Integrity Constraints

Assume the tables are:

STUDENT(SID, Name, DOB, DeptID)DEPARTMENT(DeptID, DeptName)

1. **INSERT Statement**
    1. Used to add a new student.
    2. SID must be unique and DeptID must exist in the DEPARTMENT table.

INSERT INTO STUDENT (SID, Name, DOB, DeptID)

VALUES (101, 'Anu', '2005-06-15', 10);

2.UPDATE Statement

- Used to modify existing student details.
- The new `DeptID` must satisfy the foreign key constraint.

UPDATE STUDENT

SET Name = 'Ananya', DeptID = 10

WHERE SID = 101;

3.DELETE Statement

- Used to remove a student record.
- The specified `SID` must exist.

DELETE FROM STUDENT

WHERE SID = 101;

9.Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

Tuple Relational Calculus (TRC)

Assume:

EMPLOYEE(EmpID, EmpName, DeptName, Salary)

Query: Find all employees who work in the IT department and earn more than 50,000.

$$\{e \mid EMPLOYEE(e) \wedge e.DeptName='IT' \wedge e.Salary>50000\}$$

Explanation

- e → Represents a tuple (employee record).
- EMPLOYEE(e) → e belongs to the Employee relation.
- e.DeptName = 'IT' → Employee works in IT.
- e.Salary > 50000 → Employee earns more than 50,000.
- The result contains **all employee records satisfying both conditions**.

Q10.Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

irline Reservation System – Relational Schema

A simple schema can contain the following tables:

1. **PASSENGER**
    1. `PassengerID` – Primary Key
    2. `Name`
    3. `Phone`
    4. `Email`
2. **FLIGHT**
    1. `FlightID` – Primary Key
    2. `FlightNo`
    3. `Source`
    4. `Destination`
    5. `DepartureTime`
3. **AIRLINE**
    1. `AirlineID` – Primary Key
    2. `AirlineName`
4. **RESERVATION**
    1. `ReservationID` – Primary Key
    2. `PassengerID` – Foreign Key
    3. `FlightID` – Foreign Key
    4. `BookingDate`
    5. `SeatNo`
5. **AIRCRAFT**
    1. `AircraftID` – Primary Key
    2. `AircraftName`
    3. `Capacity`
    4. `AirlineID` – Foreign Key